

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Towards AI · Following

I Built an Agentic Coding Harness Across Three CLI hosts. Here's How It Works

8 min read · 2 hours ago



Caspar Bannink

Follow



Listen



Share

... More

This article is a work in progress. I will keep updating it as the kit evolves.

Last spring, an agent rebuilt my email-templating system for the third time. Same logic, different repo, no memory of the previous two attempts. The speed of vibecoding was getting taxed by the cost of the agent forgetting what I had already shipped.

The fix was not a smarter prompt. It was a smaller vocabulary.

I built [agentic-coding-kit](#). 17 agents, 12 slash commands, 35+ PowerShell tools, a Playwright visual pipeline, and a memory system that compounds across sessions. It installs across Claude Code, OpenCode, and GitHub Copilot CLI from one canonical source. This post explains the full architecture.

The core idea: slash commands are workflows, agents are leaves

Anthropic's own docs say it plainly: “the main agent writes code, edits files, and runs commands itself, dispatching subagents in the background.” Every successful open-source kit I surveyed follows this pattern. I learned it the hard way by building 21 wrapping orchestrator agents and killing them all.

The kit has 12 commands: 8 workflows plus 4 utility commands (`/bootstrap-harness`, `/kit-init`, `/wiki-init`, `/analyze`).

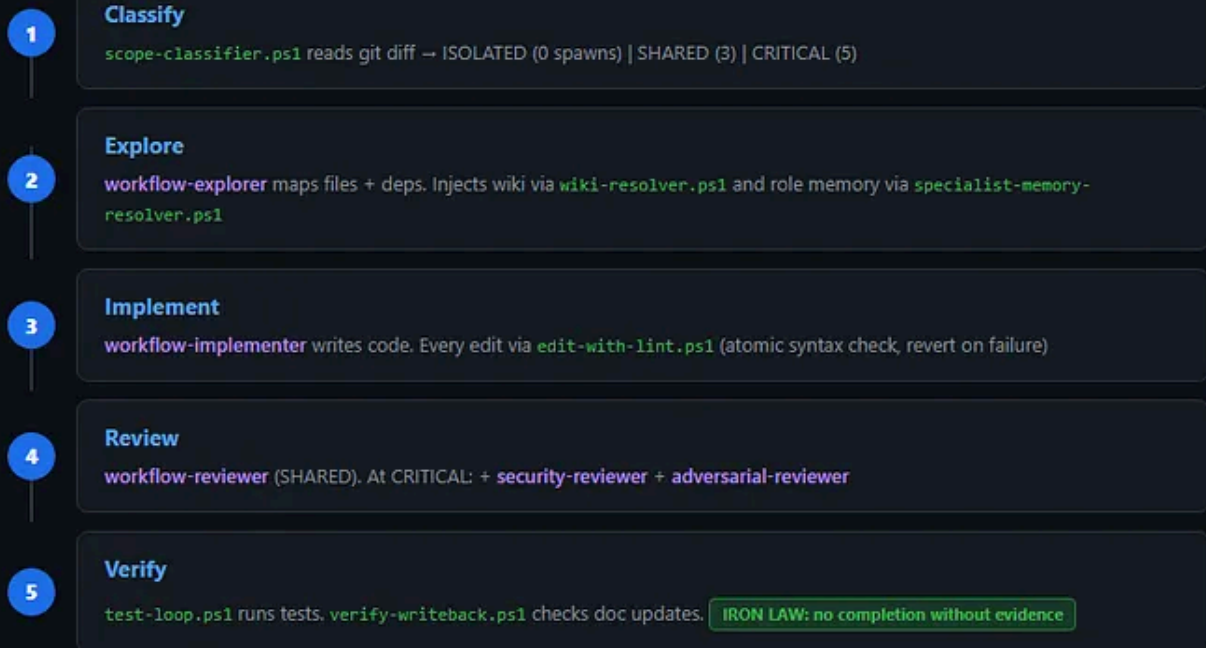
Command	What it does
<code>`/build`</code>	Scope, explore, implement, review, verify
<code>`/review`</code>	Surface, specialist, adversarial, false -positive verifier
<code>`/investigate`</code>	Hypothesis-driven debugging, evidence collection
<code>`/plan`</code>	Clarify scope, map files, stop for approval
<code>`/refactor`</code>	Principle-driven restructuring with consequence tracing
<code>`/redesign`</code>	Aesthetic lock, capture, per-component design, visual dif
<code>`/security-review`</code>	Adversarial audit by attack class
<code>`/analyze`</code>	Multi-angle research with claim verification
<code>`/bootstrap-harness`</code>	Detect repo conventions, write to conventions.md
<code>`/kit-init`</code>	Initialize <code>.kit/context/</code> in a new repo
<code>`/wiki-init`</code>	Bootstrap <code>.wiki/</code> from code evidence

These commands run differently on each host. In Claude Code and OpenCode, they run as slash commands where the main session acts as orchestrator and spawns leaf agents via the Task tool. In Copilot CLI (which does not support custom slash commands or orchestrator-style spawning), each workflow is a shell script: `kit-build.sh chains copilot --agent workflow-explorer -p "..."` followed by `copilot --agent workflow-implementer -p "..."` as direct sequential commands. Different host architecture, same leaf agents, same phases.

How the workflows actually work

Every workflow starts the same way: `scope-classifier.ps1` reads the git diff and classifies the task as ISOLATED (single file, 0 spawns), SHARED (multi-file, 3 spawns typical), or CRITICAL (auth, schema, migrations: 5 spawns, adversarial pass included). This determines the ceremony tier before anything else runs.

/build Pipeline



/build is the most common. Phase 1 spawns `workflow-explorer` to map files, trace dependencies, and return a structured brief (with wiki and specialist memory injected via resolvers). Phase 2 spawns `workflow-implementer`, which writes code using `edit-with-lint.ps1` for every change (atomic syntax check, revert on failure). Phase 3 spawns `workflow-reviewer` (one reviewer, not seven, the adversarial pass only runs at CRITICAL tier). Phase 4 runs `test-loop.ps1` and `verify-writeback.ps1`. If verification fails, the session cannot claim completion. This is the Iron Law.

/investigate runs hypothesis-driven debugging. It collects symptoms, generates ranked hypotheses, runs the cheapest test first, collects evidence, and outputs a build brief. The output is a diagnosis, not a fix. If the diagnosis warrants code changes, it hands off to `/build`.

/review fans out to specialist reviewers in parallel: `code-quality-reviewer`, `security-reviewer`, `api-reviewer`, `testing-reviewer`, and optionally `performance-reviewer`,

`adversarial-reviewer`, `data-migration-reviewer`. A `false-positive-verifier` filters noise before the final report. Findings update `handoffs.md` (active items), `memory.md` (recurring patterns), and `reflections.md` (workflow improvements).

`/analyze` is for research, not code. It spawns 4 explorers in parallel (`architecture-explorer` , `surface-explorer` , `risk-explorer` , `ops-explorer`), then 4 theorists (`pragmatist` , `skeptic` , `security-reliability` , `product-wedge`), then a `claim-verifier` that checks claims against actual code. Useful for "what is this codebase and where would I extend it."

`/redesign` activates the Playwright visual pipeline (covered below). `/refactor` does principle-driven restructuring with consequence tracing. `/security-review` runs adversarial audit by attack class. `/bootstrap-harness` detects git workflow, architecture preferences, and PR review conventions from a repo and writes them to `.kit/context/conventions.md` so other agents implement per the repo's actual style.

`goal-orchestrator` is different from all of these. It is the only agent that runs as a true autonomous loop. You give it a goal ("make all tests pass", "deploy to staging"), and it iterates until the goal is reached or it hits the convergence cap (6 iterations). It has mechanical stuck detection (same file, same line, same rule blocking for 3 iterations triggers rollback), an empty-diff watchdog, and a rollback gate that reverts iterations that worsen verification. It outputs a machine-parseable `GOAL_STATUS` first line. This is for autonomous work, not interactive sessions.

The three agent layers

Layer	Count	Purpose
Workflow agents	5	Generic transport: <code>`workflow-explorer`</code> , <code>`workflow</code>
Specialist agents	10	Domain experts spawned on demand: <code>`code-quality-r</code>
Orchestrators	2	<code>`goal-orchestrator`</code> (autonomous convergence loops

17 Agents Across Three Layers

WORKFLOW AGENTS — GENERIC TRANSPORT

5

workflow-explorer
maps files, traces deps

workflow-implementer
writes code

workflow-reviewer
checks correctness

workflow-skeptic
challenges plans

workflow-ui-qa
tests UI flows

SPECIALIST AGENTS — DOMAIN EXPERTS, SPAWNED ON DEMAND

10

code-quality-reviewer
lint-level checks

security-reviewer
trust boundaries, auth

modularity-expert
coupling, cohesion

adversarial-reviewer
production failures (CRITICAL only)

final-verifier
last verification pass

qa-agent
edge case testing

spec-agent
formalises requirements

playwright-navigator
browser automation

ux-driver
structural critique

ui-driver
visual polish

ORCHESTRATORS — AUTONOMOUS LOOPS & VERDICTS

2

goal-orchestrator
converge until goal met (cap 6 iters, stuck detection, rollback)

pr-reviewer
holistic PR verdict (APPROVE / REQUEST_CHANGES / COMMENT)

Each specialist has its own memory at `.kit/context/agent-memory/{role}.md`, lazy-loaded by `specialist-memory-resolver.ps1`. Only the memory relevant to the spawned role gets embedded. This is the difference between an agent that burns 8000 tokens orienting itself and one that starts working immediately.

The Playwright design pipeline

When `/redesign` or a frontend `/build` runs, the kit activates a visual pipeline that separates aesthetic direction from code.

Playwright Design Pipeline



1. Aesthetic Lock

aesthetic-director proposes 2-3 named aesthetics. User picks one. Writes locked typography, palette, density, motion rules.
Reads: `.wiki/features.md` → Writes: `DESIGN.md`



2. Capture

playwright-explorer starts dev server, captures numbered PNGs at 2x scale (3840x2160). Multi-fallback selectors. Never edits code.
Reads: `screen-flows.yaml` → Outputs: `001_dashboard_hero.png`, `002_...`



3. Structure Critique

ux-driver judges hierarchy, scannability, cognitive load, a11y, empty/error/loading states against `DESIGN.md`.
Returns: `structure_ok = true / false`

⚠ If `structure_ok = false` → ui-driver is BLOCKED. Restructure plan returned first.



4. Visual Polish

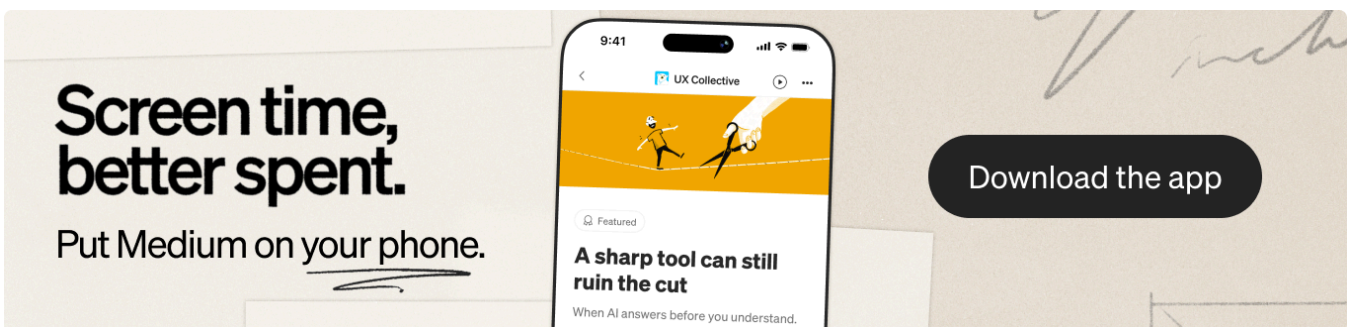
ui-driver judges color, spacing, typography, AI-slop patterns. ONE change at a time. Code updated, playwright recaptures.
Rule: `emit single change` → `verify screenshot` → `next change`



5. Visual Diff

visual-diff.ps1 pairs before/after PNGs. Pixel-level diff with ImageMagick. Regression threshold: 5%.
Outputs: `*_diff.png` + JSON report (`per-image diff_score`, `regression status`)

Step 1: Aesthetic lock. `aesthetic-director` reads `.wiki/features.md` (the product context), proposes 2-3 named aesthetics (Swiss Minimalism, Glassmorphism, etc.), and writes `DESIGN.md` with locked typography, palette, density, and motion rules. This creates consensus on what "looks right" before any code runs.



Step 2: Capture. `playwright-explorer` reads `.agents/screen-flows.yaml` (screens, auth, selectors with multi-fallback arrays), starts the dev server, and captures numbered PNGs at 2x device scale (3840x2160). It never edits code.

Step 3: Structure critique. `ux-driver` reads the screenshots against `DESIGN.md` and judges hierarchy, scannability, cognitive load, ally, and empty/error/loading states. Returns `structure_ok=true/false`. If false, `ui-driver` is blocked and a restructure plan is returned first.

Step 4: Visual polish. `ui-driver` (only if `structure_ok=true`) judges color, spacing, typography, and AI-slop patterns. It emits one change at a time, the code is updated, `playwright-explorer` recaptures, and `visual-diff.ps1` pairs before/after screenshots

Open in app ↗

≡ Medium



b

Wiki and memory: how agents remember across sessions

The kit splits context into two systems that compound across sessions.

Wiki (`.wiki/`) is the repo's structural documentation: `index.md`, `architecture.md`, `codebase.md`, `features.md`, and per-section pages in `sections/`. Each section has size budgets (150 lines max) and a standard template: purpose, key files, inputs, outputs, interactions, non-obvious notes. `wiki-resolver.ps1` lazy-loads only sections whose key files overlap with the current task. Agents never bulk-load the wiki tree.

Memory (`.kit/context/`) is the operational state: `memory.md` for durable repo facts. `handoffs.md` for what was built, what tests passed, what is next. `reflections.md` for suggested workflow improvements. `agent-memory/{role}.md` for role-specific guidance ("in this repo, X is intentional, skip flagging").

When I sit down Tuesday wondering what I worked on Friday, I open the handoff and have my answer in 30 seconds: task description, files touched, gates marked, what is left.

The self-improvement loop

`post-session.ps1` runs at session end with failure detectors: verification gate unmarked? Writeback skipped? Tests not logged? Each writes a structured entry to `reflections.md`.

`auto-apply-reflect.ps1` processes these entries in tiered safe buckets. Specialist memory accretion (3+ consistent false-positives auto-append to agent-memory). `conventions.md` refinements. Spawn-budget tweaks. These auto-apply. Slash

command body changes and tool logic rewrites are gated for manual `/reflect` approval. Every auto-applied change is snapshotted and audit-logged.

```
proposal-2026-05-04-tier-overridden-downward.md -- ~/.agents/proposals/

1 # harness-propose: tier-overridden-downward (recurring)
2
3 > 5 occurrences total, 4 in last 30 days
4 > Generated: 2026-05-04T16:42:00Z
5
6 ## The recurring problem
7
8 Sessions classified as TARGETED tier are being downgraded to
9 INLINE during /build, skipping the spec-reviewer + adversarial
10 pass. The diff size and shared-surface check both flagged
11 TARGETED, but the agent shipped without the gates.
12
13 ## Suggested target files
14
15 - ~/.agents/tools/scope-classifier.ps1
16 - ~/.agents/skills/build/SKILL.md (Phase 0 freshness check)
17 - ~/.agents/tools/state-init.ps1 (tier override guard)
18
19 ## Evidence
20
21 | Session | Task | Tier |
22 |-----|-----|-----|
23 | 2026-04-12--134422 | Add /api/quota endpoint | INLINE |
24 | 2026-04-19--091508 | Refactor auth middleware | INLINE |
25 | 2026-04-22--170303 | Migrate to Convex v3 | INLINE |
26 | 2026-04-28--084421 | Add 2FA flow | INLINE |
27 | 2026-05-02--113207 | Update payment webhook | INLINE |
28
29 ## Risks of changing this
30
31 Tightening the override would block legitimate INLINE downgrades
32 on truly trivial mechanical changes (rename, format, single-line).
33 Keep the manual override flag; just make it require an explicit
34 reason field that gets recorded in workflow-evidence.json.
35
36 ## Decision
37
38 [ ] accept [ ] reject [ ] defer

The kit telling me what to fix in itself.
```

One canonical, three hosts

Each CLI host wants agent files in a different format and silently rejects the others. Claude Code rejects BOM-prefixed files. OpenCode rejects Claude’s `tools: A, B, C` comma-string. Copilot rejects Unicode in descriptions over 300 characters.

The kit maintains one canonical source at `~/.agents/global-instructions.md`. `sync-all-hosts.ps1` distributes it into each host's instruction file between markers, preserving anything outside the markers. Per-host sanitizers translate agent files at install time. All 17 agents load on all 3 hosts from one shared definition.

The tools that enforce discipline

35+ PowerShell scripts take discipline out of the prompt and put it in code.

Tool	What it enforces
<code>`edit-with-lint.ps1`</code>	Syntax check on every edit, atomic revert on
<code>`test-loop.ps1`</code>	Marks verification gate on pass, detects stuc

`verify-writeback.ps1`	Checks docs updated alongside code changes
`scope-classifier.ps1`	Determines ceremony tier from git diff
`frontend-detector.ps1`	Flags greenfield UI for aesthetic-director
`wiki-resolver.ps1`	Lazy-loads only relevant wiki sections into a
`specialist-memory-resolver.ps1`	Injects role- specific repo memory into subage
`visual-diff.ps1`	Pixel-level before/after comparison with regr
`auto-apply-reflect.ps1`	Closes the self-improvement loop with tiered

The bash-dispatcher hook enforces at the protocol layer: dangerous filesystem ops blocked, git push --force on main blocked, git commit gated on verification evidence. Test success auto-marks the verification gate. The agent does not need to remember any of these rules. The harness does.

What is next

The kit runs on Claude Code today. In the next article, I will write about the hybrid setup I am testing: Claude Code as orchestrator, OpenCode Go at \$10/month with DeepSeek V4 API keys for the fan-out layer. DeepSeek V4 Flash at \$0.14/\$0.28 per million tokens is 35x cheaper than Opus on input. A `/build` with 3 reviewer spawns costs under \$0.01.

Fork or clone: github.com/CBannink/agentic-coding-kit. Open an issue if something breaks.

I am Caspar Bannink, founder of HomeScout (AI rental search for Dublin) and Bannink Software Development.

Side project: homescout.io

Personal LinkedIn: linkedin.com/in/caspar-bannink-719440217

HomeScout LinkedIn: linkedin.com/company/homescout-io

Repo: github.com/CBannink/agentic-coding-kit

[Agentic AI](#)[Claude Code](#)[Open Code](#)[AI](#)[AI Agent](#)



Following

Published in Towards AI

138K followers · Last published 2 hours ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>



Follow

Written by Caspar Bannink

26 followers · 10 following

Writing about theoretical and applied AI, check out my startup at homescout.io where I leveraged AI to make the rental search in Dublin - Ireland easier

